

Ink-Edit: Interactive color-constrained textures editing

Charline Grenier
ICube, Strasbourg University, CNRS
France
charline.grenier@unistra.fr

Pooran Memari
CNRS, LIX, Ecole Polytechnique, Inria
France
memari@lix.polytechnique.fr

Basile Sauvage
ICube, Strasbourg University, CNRS
France
sauvage@unistra.fr

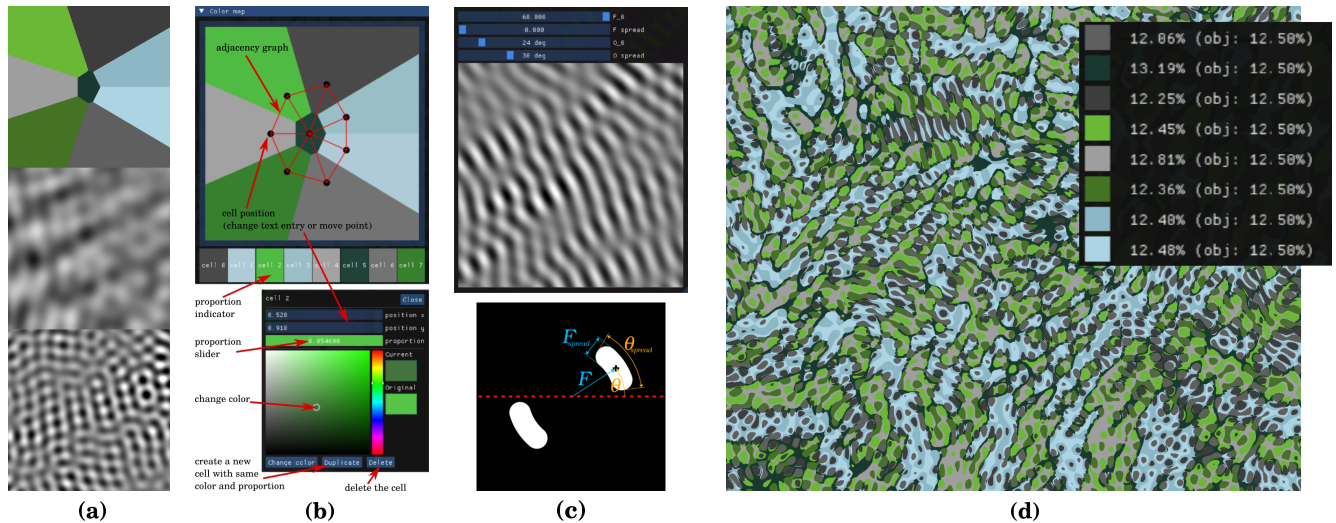


Figure 1: Ink-Edit interface offers an editing tool for stochastic micro-pattern textures. The underlying procedural generation model is based on a color map and Gaussian noises (a). The user can prescribe the objective proportions of the colors and further edit the adjacency interactively (b). The corresponding space of textures can be explored via the spectral edition of the procedural Gaussian noises (c). The output and resulting color proportions are shown in (d).

Abstract

Procedural textures are able to generate large and detailed textures on-demand, at rendering time, with minimal memory usage. They are defined by parallel algorithms, which are easily integrated into modern graphics hardware. As a drawback, they may be difficult to control, namely in an interactive and user-friendly manner. In this paper, we define an interactive tool to control the parameters of the procedural model introduced by Grenier et al. [Grenier et al. 2022], which is based on a noise vector field (Figure 1 (a), bottom and middle), and a color map (Figure 1 (a), top). In our interactive tool, the noise is controlled through a small set of parameters in the spectral domain. To control the color map, the user first prescribes the colors (inks) and their proportions (ink volumes in the final result), and then adjusts interactively the adjacency between colors. This is done through a new constrained optimal transport point of view: the colors in the map are regarded as cells of a weighted Voronoi diagram, and the noise distribution is regarded as a probability

measure. The ink volumes are automatically enforced as hard constraints of an optimization process during the interactive session, while the user focuses on high-level control over the adjacency of the colors.

CCS Concepts

• Computing methodologies → Image processing; Texturing; Graphics input devices.

Keywords

Texture Synthesis, Micro-pattern, Voronoi diagrams, Optimal transport, Capacity constraint optimisation, Gabor noise, Pattern generation, Interactive texture editing, Procedural modeling.

ACM Reference Format:

Charline Grenier, Pooran Memari, and Basile Sauvage. 2024. Ink-Edit: Interactive color-constrained textures editing. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3697991>

1 Introduction

Textures are commonly used in 3D graphics as they are efficient in adding visual details on the surface of 3D objects. Large virtual scenes challenge the processes for generating, storing, and rendering huge amounts of textures. Procedural texture generation offers

several advantages: textures are generated on-demand using fast parallel algorithms that are memory efficient. However, a significant downside is that they are often difficult to control due to the many non-intuitive parameters. Managing patterns that exhibit both local structures (e.g., strong color variations) and stochastic shapes has been a challenge in real-time applications.

Grenier et al. [Grenier et al. 2022] recently proposed a versatile model for such patterns. However, this model is hard to control, limiting its use by artists in industrial creative pipelines. Indeed, the model is parameterized by the power spectral density (PSD) of Gaussian noises, and by a 2D color map, each one having tens of thousands of degrees of freedom. We aim at reducing this number to only a few tens of intuitive parameters, along with an interactive tuning tool.

Texture model. Our framework is based on the micro-pattern texture model introduced in [Grenier et al. 2022], illustrated in Figure 2. In this model, we are given a noise vector field N , defined from $\mathbb{R}^2$ to an interval $\mathbb{X} \subset \mathbb{R}^2$. Moreover, we are provided with a look-up table H defined on $\mathbb{X}$, referred as "color map" or "transfer function", which associates a color value to every noise value. The texture T is then defined through the following composition

$$T(\mathbf{u}) := H(N(\mathbf{u})) = H(N_x(\mathbf{u}), N_y(\mathbf{u})), \quad (1)$$

where $\mathbf{u} \in \mathbb{R}^2$ is a texture coordinate, and can be generated by parallel stochastic algorithms [Grenier et al. 2022]. In practice, T and N are considered over a rectangular domain $\mathbb{U} \subset \mathbb{R}^2$, representing a texture canvas.

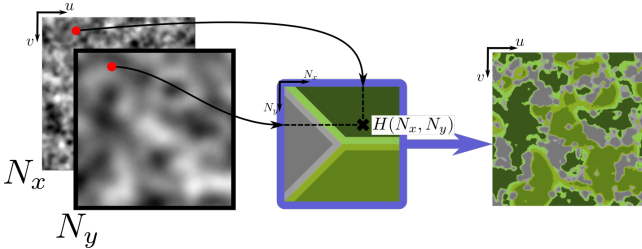


Figure 2: The noise vector field (on the left) is composed with the color map (at the center) to obtain the result texture (on the right). Texture coordinates are represented by $\mathbf{u} = (u, v)$.

In this model, the resulting texture appearance arises from the complex interaction between N and H , which lacks user control. Therefore, we target an interactive approach that allows users to set certain meaningful parameters while intuitively adjusting others.

Methodology. The noise N is managed through the power spectral density (PSD) of its two components, using a well-established technique that requires only four parameters per PSD (Section 4). Essentially, these parameters specify a frequency interval and an orientation interval for each noise component. The most challenging part of the process is creating the color map. Empirically, we found that compact color cells in H are intuitive, with three-color junctions being the most common in natural textures. Additionally, we identified three key aspects to control intuitively: the list of colors in the result, the proportion of each color, and the spatial

relationships between colors. In view of these objectives, we have opted for planar Voronoi diagrams to compute H . They tile the space $\mathbb{X}$ into compact cells, with three-cell junctions in non-degenerate cases. The colors of cells directly control the colors in T . Since N is continuous, only neighboring cells in H produce neighboring colors in the result, so the topology of the diagram controls the adjacency of colors. To control the proportions of colors, we leverage the capacity constrained setting linking geometric structures, namely weighted Voronoi diagrams, to an optimal transport setting [De Goes et al. 2012]. This framework provides a control on the cell volumes, which is reflected in the final resulting texture, as detailed in section 3. This leads to an intuitive texture editing pipeline with a user-friendly interface, where the proportion of each color in the final texture can be prescribed while the texture gets edited interactively (Section 4).

2 Related Work

Procedural Texture Synthesis. Procedural texture synthesis has evolved significantly, leveraging procedural noise generation techniques for creating, in real-time, realistic micro-patterns that include an element in randomness. From the seminal published in the '80s and '90s (e.g. by Perlin, van Wijk, or Worley), several methods contributed to this research field [Lagae et al. 2010]. The processing of local structures, i.e. strong color variations, has been an enduring challenge. Techniques based on point processes are efficient for cellular patterns [Guehl et al. 2020; Worley 1996]. Another approach consists in applying discontinuous transfer functions to a smooth Gaussian field, such as a color map on a real field [Heitz et al. 2013] or a profile on a phase field [Tricard et al. 2019]. These two methods are particular cases of the more generic model of [Grenier et al. 2022], which provides real-time generation and rendering of structured micro-patterns. The expressiveness of the model is due to the rise in dimension of the transfer function (a.k.a. color map). Our work provides this model with a tool to explore and control the result through a reduced set of parameters that can be adjusted interactively.

Interactive Texture Design. Procedural texture generation is often difficult to control due to the numerous and non-intuitive parameters of the models. To overcome this difficulty, a first approach consists in so-called *by-example* methods: the model's parameters are inferred from a small example, given as input. This approach is used by techniques like tiling and blending [Heitz and Neyret 2018], Gabor noise by-example [Galerie et al. 2012], and multiple kernels noise [Gilet et al. 2012]. However, the user is unable to easily explore texture patterns or adjust the parameters to achieve a specific artistic goal.

More recently, learning-based methods have been employed to retrieve and manipulate complex procedural graphs [Guerrero et al. 2022; Hu et al. 2023]. These methods generate graphs with numerous nodes and hundreds of parameters. Our approach is complementary, as the model of Grenier et al. [Grenier et al. 2022] as a whole is one single possible node in these graphs –not to be confused with our adjacency graph hereafter. We provide a way to control this node precisely while reducing the number of parameters.

Interactive modification of the resulting texture is highly valuable for artists aiming to adjust a procedural model. However, few procedural noise methods offer interactive parameter manipulation. One noticeable technique is the spectral manipulation of procedural Gabor noise [Lagae et al. 2009], which we adapt to control the Gaussian noises. However, we still lack a solution for the color maps. In this paper, we propose a novel framework that allows for the interactive modification of both the noise and the color map separately, using an adjacency graph representation and a geometric optimization formulation, as shown in Figure 1 (b). This way, the user can interactively manipulate the color configuration to efficiently create procedural textures with specified color proportions.

3 Color map design & geometric optimization

We will regard the micro-pattern generation procedure from a constrained optimal transport point of view. The probability density function of the noise N is identified with a probability measure $\rho(\mathbf{x})$. Let us recall the image N of the texture canvas $\mathbb{U} \subseteq \mathbb{R}^2$, defined as

$$N(\mathbb{U}) = \{(N_x(\mathbf{u}), N_y(\mathbf{u})) \mid \mathbf{u} \in \mathbb{U}\}. \quad (2)$$

For any values $\mathbf{x} \in N(\mathbb{U})$, the inverse image can be defined as

$$N^{-1}(\mathbf{x}) = \{\mathbf{u} \mid (N_x(\mathbf{u}), N_y(\mathbf{u})) = \mathbf{x}\}. \quad (3)$$

This leads to a probability measure over the image of N

$$\rho(\mathbf{x}) = \frac{\mu(N^{-1}(\mathbf{x}))}{\mu(\mathbb{U})}, \quad (4)$$

where μ denotes the Lebesgue measure of sets in $\mathbb{R}^2$. In other words, $\rho(\mathbf{x})$ is the probability of using $\mathbf{x} = N(\mathbf{u})$ in the generation of the final texture. Note that ρ is a normalized Gaussian function, whose mean and covariance matrix are determined by the PSD of N , and is used as the probability measure for our optimal transport setting. This probability density is computed from the noise model and used for an optimal design of the color map under the user's constraints.

Constraints formulation. Let $\{c_i\}_{i=1}^n$ be the palette of colors prescribed by the user with respective percentage proportions $\{m_i\}_{i=1}^n$, which sum up to 1. We aim at a partitioning of the color map H into regions v_i associated to each color c_i , so that $m_i = \int_{v_i} \rho(\mathbf{x}) d\mathbf{x}$, $\forall i$.

As will be shown, a convenient partitioning for such constrained optimization, is *Weighted Voronoi diagram*, defined as follows. Let $\{\mathbf{x}_i\}_{i=1}^n$ be n points in $\mathbb{R}^2$, associated to scalar weights $\{w_i\}_{i=1}^n$. The weighted Voronoi cell v_i , associated to each $\mathbf{x}_i$, is defined as

$$v_i = \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x} - \mathbf{x}_i\|^2 - w_i \leq \|\mathbf{x} - \mathbf{x}_j\|^2 - w_j, \forall j \neq i\}. \quad (5)$$

In our context, we will represent each color c_i in the palette by a representative seed point $\mathbf{x}_i$ on the color map H . We will find the optimal positions and weights so that the partitioning into cells v_i matches the prescribed colors' proportions. More formally, for provided probability density ρ and prescribed proportions m_i , find points and weights so that $\{v_i\}_{i=1}^n$ satisfy the following constraints:

$$m_i = \int_{v_i} \rho(\mathbf{x}) d\mathbf{x}, \quad \forall i. \quad (6)$$

This optimization problem is known as a capacity constrained optimal transport framework, which can be efficiently solved by convex optimization [Aurenhammer et al. 1998]. We refer to [De Goes

et al. 2012] for more details. To find the optimal partitioning, we minimize the following energy:

$$E_V = \sum_{i=1}^n \int_{v_i} \rho(\mathbf{x}) \|\mathbf{x} - \mathbf{x}_i\|^2 d\mathbf{x}, \quad (7)$$

under the constraints (6), which is indeed equivalent to minimizing

$$E_V + \sum_i w_i \left(\int_{v_i} \rho(\mathbf{x}) d\mathbf{x} - m_i \right) \quad (8)$$

with respect to $\mathbf{x}_i$ and w_i , where the weights w_i act as Lagrange multipliers. This equivalence is proven in [De Goes et al. 2012], where closed form formulas are derived for updating both the weights and positions. The optimization process starts with zero weights and initial positions chosen by the user. This is followed by an alternative optimization of weights w_i and positions $\mathbf{x}_i$. This leads to a color map which fulfills the target color proportions.

4 Interactive interface & noise features

In an interactive session, the user can select and adjust, in turn:

- (a) Palette of colors $\{c_i\}_{i=1}^n$ and their proportions $\{m_i\}_{i=1}^n$.
- (b) Positions of seed points $\mathbf{x}_i$, for color neighborhoods control.
- (c) Noise spectrum features for texture pattern exploration.

For each user interaction, the resulting texture is calculated and shown in real-time [Grenier et al. 2022]. When (a) or (b) are modified, the Voronoi tiling is recomputed using the optimization of Section 3. The interface widgets are shown in Figure 1 (b) and (c), and are detailed in the following.

(a) *Colors palette and proportions.* To update the palette, the user can add, delete, or modify colors using a color-picking interface (Figure 1 (b), bottom). The proportion of each color is controlled with a slider. Based on these constraints, the seed points positions and the weight of the cells are adjusted with the optimization process developed in Section 3. The distribution of the colors in the final texture is represented by a multi-color bar below the color map, where each rectangle's width corresponds to the color proportion.

(b) *Colors neighborhoods.* The colors adjacency graph, which can be visualized on top of the color map (in red Figure 1 (b)), encodes the relationships between color cells. Cell configuration is controlled interactively by the position of seed points, which can be moved by dragging and dropping in the interface. Their positions and weights are optimized at each iteration to fulfill proportions constraints, as in Figure 1.

(c) *Spectrum of the noise.* The two components, N_x and N_y , are manipulated individually. As Gaussian noises, they are fully described by their Power Spectral Distribution (PSD), which is symmetric. Our interface mimics an interaction method proposed by [Lagae et al. 2009], exploring part of the noise space using three intuitive ideas: main orientation, anisotropy, and frequency band-pass. It operates in the frequency domain to define a binary PSD, where each frequency is assigned either full energy (white) or zero energy (black). In this context, only four parameters are adjusted to edit the noise and the resulting pattern, as illustrated in Figure 1 (c): the main frequency F and frequency spread F_{spread} control the bandpass, the main orientation θ determines the global orientation of the noise, and the orientation spread θ_{spread} allows tuning

from pure anisotropy ($\theta_{spread} = 0$) to a perfectly isotropic pattern ($\theta_{spread} = \pi$).

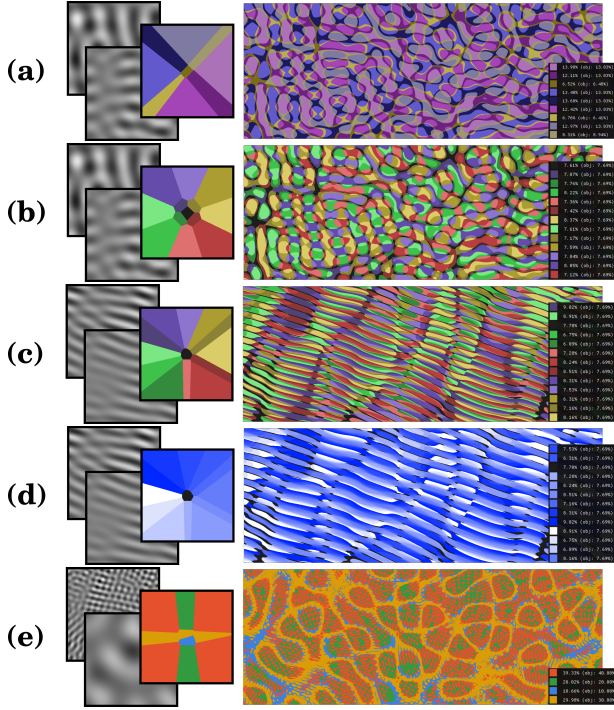


Figure 3: Variety of texture patterns generated via the proposed interactive interface. In rows (a) and (b), the textures share identical input noises but differ in color maps. Rows (c) and (d) have matching noises and cell positions within the color map, with differences only in cell colors. Row (e) shows a color map containing several cells with same color (four red cells, three yellow and two green cells, and one blue cell).

Performances. The interface runs at 60 fps on Nvidia GTX 670, and 100 fps on AMD RX 1700 XT. The time of the optimization depends on the resolution of the color map, and we measured up to 1 second per optimization. As can be seen in the black insets in Figures 1 and 3, the measured final proportion slightly differs from the prescribed objective. The final proportions are measured in the result texture by counting the pixels for each colors. In our tests, the relative error is 3% on average. This is due to the approximations made in computing the probability measure ρ . In theory, it is a Gaussian distribution, characterized by the mean and variance of the vector noise. However, the actual measure is based on the discrete histogram of the result, which is not a perfect Gaussian because the result image contains a finite number of pixels. Note that the error decreases as the noise resolution increases, and –in theory– it tends to zero as the resolution tends to infinity.

5 Perspectives and application scenarios

Controlling material proportions on a surface is crucial in various domains. In particular, the control over color proportions in the texture generation can be used for data augmentation for datasets

in which a specific style or ink proportions appear naturally. This justifies the potential impact of interactive tools such as ours, providing computational control over specific quantities. For instance, we are investigating the use of our model in histological medical image augmentation, providing a parameterized control over the colors' proportion representing different tissues or cellular pathology. Moreover, although this paper demonstrates results only for surface textures, our algorithm is also applicable to volumetric texture generation by using volumetric noise instead of surface noise, while using the same easy-to-control 2D color map. This straightforward extension is motivated by the need to generate heterogeneous materials composed of various micro-materials with prescribed proportions. Finally, our approach drastically reduces the number of parameters needed to control the model, which is a major advantage for its use in an artistic pipeline. Its use as a node in procedural graphs would be very interesting.

Acknowledgments

This work was supported by the project ReProcTex (ANR-19-CE33-0011-01) and by the ITI IRMIA++, as part of the ITI 2021-2028 program of Strasbourg University, CNRS and Inserm, supported by IdEx Unistra (ANR-10-IDEX-0002) and by SFRI-STRAT'US project (ANR-20-SFRI-0012) under the framework of the French Investments for the Future Program. Part of this work was done during the second author's visit to TU Berlin, thanks to Marc Alexa's team.

References

- Franz Aurenhammer, Friedrich Hoffmann, and Boris Aronov. 1998. Minkowski-type theorems and least-squares clustering. *Algorithmica* 20 (1998), 61–76.
- Fernando De Goes, Katherine Breeden, Victor Ostromoukhov, and Mathieu Desbrun. 2012. Blue noise through optimal transport. *ACM Transactions on Graphics (TOG)* 31, 6 (2012), 1–11.
- Bruno Galerne, Ares Lagae, Sylvain Lefebvre, and George Drettakis. 2012. Gabor noise by example. *ACM Transactions on Graphics (TOG)* 31, 4 (2012), 1–9.
- Guillaume Gilet, Jean-Michel Dischler, and Djamchid Ghazanfarpour. 2012. Multiple kernels noise for improved procedural texturing. *The Visual Computer* 28 (2012), 679–689.
- Charline Grenier, Basile Sauvage, Jean-Michel Dischler, and Sylvain Thery. 2022. Color-mapped noise vector fields for generating procedural micro-patterns. In *Computer Graphics Forum*, Vol. 40.
- Pascal Guehl, Rémi Allegre, J-M Dischler, Bedrich Benes, and Eric Galin. 2020. Semi-Procedural Textures Using Point Process Texture Basis Functions. In *Computer Graphics Forum*, Vol. 39. Wiley Online Library, 159–171.
- Paul Guerrero, Miloš Hašan, Kalyan Sunkavalli, Radomir Měch, Tamy Boubekour, and Niloy J. Mitra. 2022. MatFormer: a generative model for procedural materials. *ACM Trans. Graph.* 41, 4, Article 46 (jul 2022), 12 pages.
- Eric Heitz and Fabrice Neyret. 2018. High-performance by-example noise using a histogram-preserving blending operator. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 1, 2 (2018), 1–25.
- Eric Heitz, Derek Nowrouzezahrai, Pierre Poulin, and Fabrice Neyret. 2013. Filtering color mapped textures and surfaces. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. 129–136.
- Yiwei Hu, Paul Guerrero, Milos Hasan, Holly Rushmeier, and Valentin Deschaintre. 2023. Generating Procedural Materials from Text or Image Prompts. In *ACM SIGGRAPH 2023 Conference Proceedings*. 1–11.
- Ares Lagae, Sylvain Lefebvre, Rob Cook, Tony DeRose, George Drettakis, David S Ebert, John P Lewis, Ken Perlin, and Matthias Zwicker. 2010. A survey of procedural noise functions. In *Computer Graphics Forum*, Vol. 29. 2579–2600.
- Ares Lagae, Sylvain Lefebvre, George Drettakis, and Philip Dutré. 2009. Procedural noise using sparse Gabor convolution. *ACM Transactions on Graphics* 28, 3 (2009).
- Thibault Tricard, Semyon Efremov, Cédric Zanni, Fabrice Neyret, Jonàs Martínez, and Sylvain Lefebvre. 2019. Procedural phasor noise. *ACM Transactions on Graphics (TOG)* 38, 4 (2019), 1–13.
- Steven Worley. 1996. A cellular texture basis function. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*. 291–294.